IUT de Toulon, département GEII
SAE3.ESE.01-02 Concevoir-Vérifier

Dossier de conception

Assistant domotique « GouGouGaGAH »
ESP32-C6 et Home Assistant

Schéma fonctionnel · Schéma structurel · Captures Home Assistant
Annexes : code Arduino, codes et configuration Home Assistant

Maximilien SEGUIER
Enseignant : M. GIBERT
Rendu le 1er novembre 2026

Sommaire

1. Schéma fonctionnel

2. Schéma structurel (KiCad)

3. Captures d'écran Home Assistant

Annexe A. Code Arduino du programme final (GouGouGaGAH_Application)

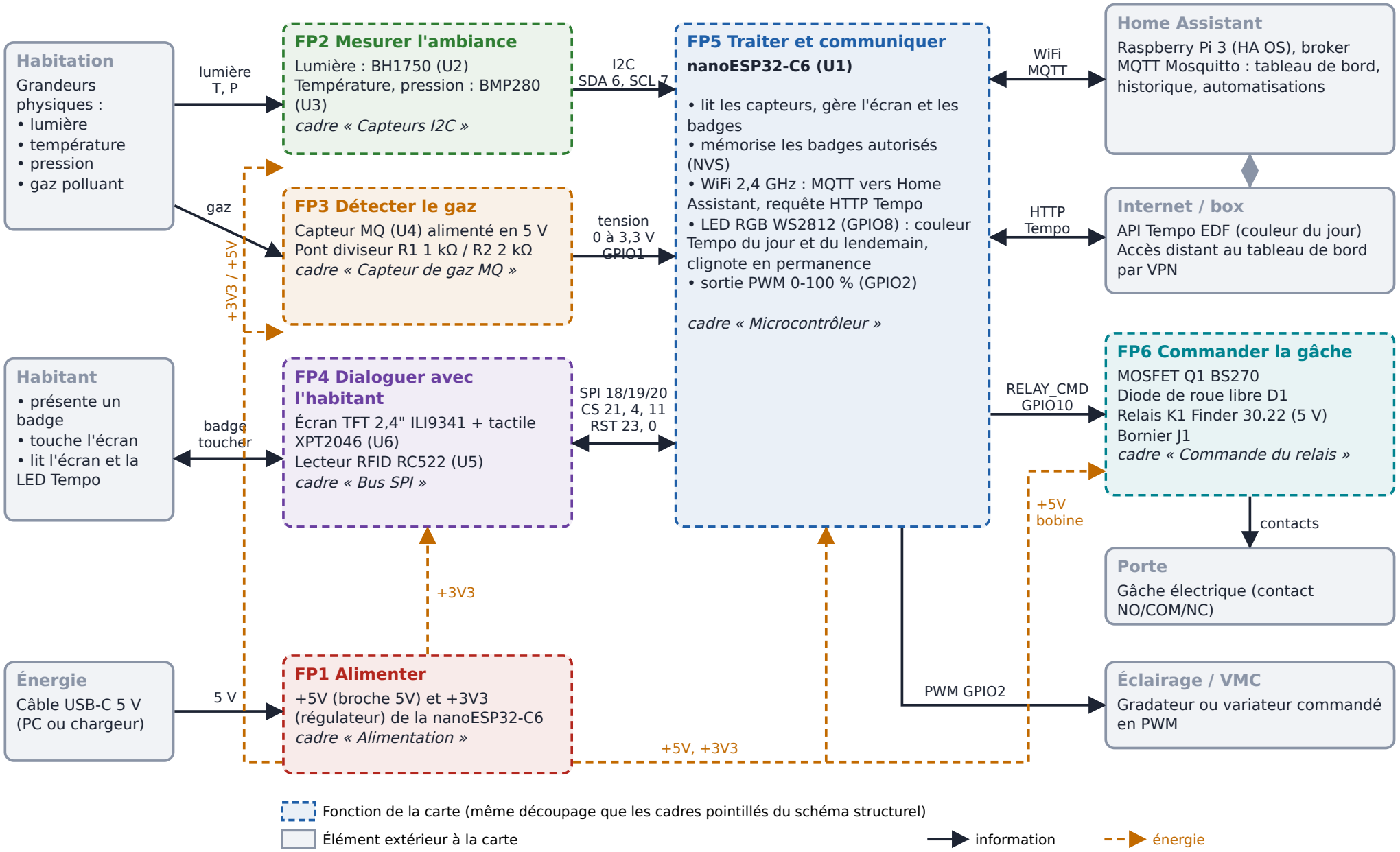
Annexe B. Codes et configuration Home Assistant

Présentation du système

L'assistant domotique affiche des informations sur l'habitation et permet de la piloter. Une carte à base de **nanoESP32-C6** mesure la lumière (BH1750), la température et la pression (BMP280) et la présence de gaz (MQ). Elle lit les badges RFID (RC522) pour commander la gâche de la porte par le relais K1, affiche les données sur un écran tactile ILI9341 et fournit une sortie PWM pour la gradation d'un éclairage ou la vitesse d'une VMC. La LED RGB de la carte indique la couleur Tempo EDF du jour et du lendemain et clignote pour attester que le programme tourne.

La carte communique en WiFi avec **Home Assistant** (Raspberry Pi 3) par le protocole MQTT : les entités sont créées automatiquement par la découverte MQTT, avec historique, tableau de bord et automatisations. Le tableau de bord est accessible sur le réseau local et à distance par VPN.

1. Schéma fonctionnel



Description des fonctions

Fonction	Composants	Rôle
FP1 Alimenter	nanoESP32-C6 (U1) : broches 5V et 3V3	Fournit le +5V (MQ, bobine du relais) et le +3V3 (capteurs I2C, écran, RFID) à partir de l'USB-C.
FP2 Mesurer l'ambiance	BH1750 GY-302 (U2), BMP280 (U3)	Mesure l'éclairement (lx), la température (°C) et la pression (hPa). Bus I2C partagé, adresses 0x23 et 0x76.
FP3 Détecter le gaz	Module MQ (U4), R1 1 kΩ, R2 2 kΩ	Donne une tension image de la concentration de gaz. Le pont diviseur ramène la sortie 0-5 V à 0-3,33 V pour l'entrée analogique GPIO1.
FP4 Dialoguer avec l'habitant	Écran TFT 2,4" ILI9341 + XPT2046 (U6), RC522 (U5)	Affiche les mesures et les menus, reçoit les appuis tactiles, lit l'identifiant des badges. Bus SPI partagé, un chip select par module.
FP5 Traiter et communiquer	nanoESP32-C6 (U1)	Exécute le programme : contrôle d'accès, menus, mémoire des badges en NVS, MQTT vers Home Assistant, lecture de la couleur Tempo, LED RGB, sortie PWM.
FP6 Commander la gâche	BS270 (Q1), D1, relais Finder 30.22.7.005 (K1), bornier J1	Le MOSFET commute la bobine 5 V du relais depuis GPIO10. D1 protège Q1 de la surtension à la coupure. Les contacts NO/COM/NC sortent sur J1 vers la gâche.

Liaisons entre fonctions

Liaison	Nature	Signaux (GPIO de l'ESP32-C6)
FP2 → FP5	numérique, I2C	SDA = GPIO6, SCL = GPIO7
FP3 → FP5	analogique	MQ_AO divisé, 0 à 3,33 V sur GPIO1 (ADC 12 bits)
FP4 ↔ FP5	numérique, SPI	SCK = GPIO18, MOSI = GPIO19, MISO = GPIO20 ; TFT_CS = 21, TFT_DC = 22, TFT_RST = 23, TOUCH_CS = 4, RFID_CS = 11, RFID_RST = 0
FP5 → FP6	logique TOR	RELAY_CMD = GPIO10, actif à l'état haut
FP5 → éclairage / VMC	PWM	GPIO2, rapport cyclique 0 à 100 %
FP5 → LED RGB	série WS2812	GPIO8 (LED intégrée à la carte)
FP5 ↔ Home Assistant	WiFi 2,4 GHz, MQTT	topics gougougagah/… (voir annexe B)
FP5 ↔ Internet	WiFi, HTTP	api-couleur-tempo.fr : couleur du jour et du lendemain

2. Schéma structurel

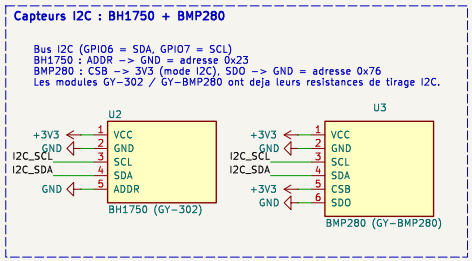
Le schéma structurel a été saisi sous KiCad 10 (projet `mSeguier/mSeguier.kicad_sch`). Il reprend le découpage du schéma fonctionnel : chaque fonction est entourée d'un cadre en trait pointillé (Alimentation, Microcontrôleur, Capteurs I2C, Capteur de gaz MQ, Bus SPI, Commande du relais). Le schéma est reproduit à la page suivante (original au format A3, en vectoriel : il peut être agrandi).

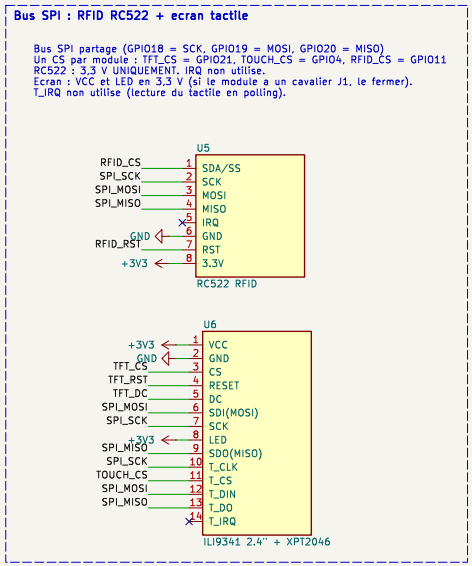
Nomenclature des repères

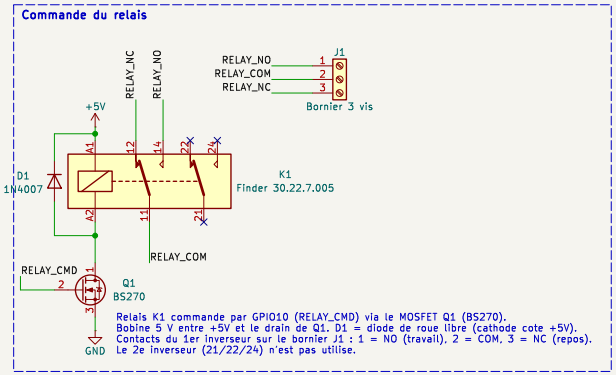
Repère	Désignation	Fonction	Remarque
U1	MuseLab nanoESP32-C6 (2 × 15 broches)	FP5, FP1	Microcontrôleur, WiFi, LED RGB WS2812
U2	Module GY-302 BH1750	FP2	ADDR à GND : adresse 0x23
U3	Module GY-BMP280	FP2	CSB à 3V3 (mode I2C), SDO à GND : adresse 0x76
U4	Module capteur de gaz MQ	FP3	Alimenté en 5 V, DO non utilisé
R1, R2	Résistances 1 kΩ et 2 kΩ	FP3	5 V × 2k / (1k + 2k) = 3,33 V max
U5	Module RFID RC522	FP4	Alimenté en 3,3 V uniquement
U6	Écran TFT 2,4" ILI9341 + tactile XPT2046 (MSP2402)	FP4	Rétroéclairage sur 3V3
Q1	MOSFET canal N BS270	FP6	Commande de la bobine depuis GPIO10
D1	Diode 1N4004	FP6	Roue libre, cathode côté +5V
K1	Relais Finder 30.22.7.005.0010, 2RT, bobine 5 V	FP6	1er inverseur utilisé
J1	Bornier à vis 3 points	FP6	1 = NO, 2 = COM, 3 = NC

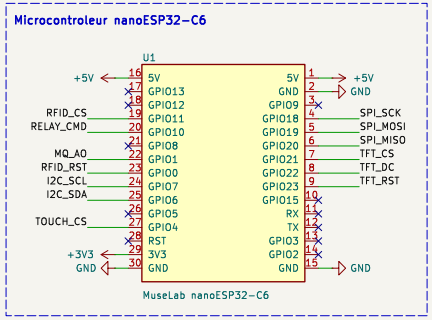
Dimensionnement de la commande du relais

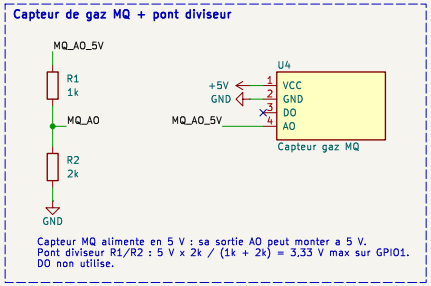
La bobine 5 V du relais Finder 30.22 consomme environ 40 mA (résistance d'environ 125 Ω). Une sortie de l'ESP32-C6 ne peut pas fournir ce courant sous 5 V : le MOSFET BS270 ($V_{GS(th)}$ inférieure à 2,5 V, I_D max 400 mA) est commandé directement en 3,3 V par GPIO10 et commute la bobine côté masse. La diode D1 en parallèle sur la bobine écoule le courant de self à l'ouverture et protège Q1.

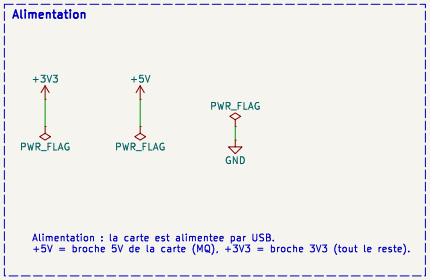












Maximilien SEGUIER
Brochage conforme a include/pinout.h du projet PlatformIO
IUT GEII Toulon – SAE3.ESE.01.02

Sheet: /
File: mSeguier.kicad_sch

Title: GouGouGaGAH – Domotique ESP32-C6

Size: A3	Date: 2026-10-21	**Rev: 2**
KiCad E.D.A. 9.0.1		Id: 1/1

3. Captures d'écran Home Assistant

Home Assistant OS est installé sur un Raspberry Pi 3 du réseau local. La carte s'y connecte en WiFi par le broker MQTT Mosquitto ; l'appareil **GouGouGaGAH** et ses entités apparaissent automatiquement (découverte MQTT).

Capture 1 : tableau de bord (mesures, porte, bouton d'ouverture, curseur d'éclairage)
(capture à insérer)

Capture 2 : courbes de tendance (historique des capteurs)
(capture à insérer)

Capture 3 : automatisations
(capture à insérer)

Annexe A. Code Arduino (GouGouGaGAH_Application)

Projet PlatformIO, framework Arduino (plateforme pioarduino), carte `esp32-c6-devkitc-1`. Code source complet du programme final, dépôt GitHub `SeguierMaximilien/GouGouGaGAH_DomotiqueESP32`, dossier `GouGouGaGAH_Application/`. Le fichier `secrets.h` (mots de passe WiFi et MQTT) n'est pas publié : seul son modèle `secrets_exemple.h` figure ici.

Fichier	Rôle
`platformio.ini`	Configuration du projet et bibliothèques
`pinout.h`	Affectation des broches
`secrets_exemple.h`	Modèle des identifiants WiFi / MQTT
`main.cpp`	Initialisation des bus et des modules, boucle principale
`module_acces`	Contrôle d'accès : badges autorisés en NVS, ouverture de la porte 3 s
`module_bh1750`	Capteur de lumière
`module_bmp280`	Capteur de température et de pression
`module_ha`	WiFi, MQTT, découverte Home Assistant, ordres reçus
`module_menu`	Menus de l'écran tactile (Capteurs, Porte, Badges, Éclairage, Infos)
`module_mq`	Capteur de gaz (valeur brute ADC)
`module_pwm`	Sortie PWM 0-100 % sur GPIO2
`module_relais`	Commande du relais K1 (GPIO10)
`module_rfid`	Lecture des badges RC522
`module_tempo`	Couleur Tempo EDF (tâche FreeRTOS) et LED RGB clignotante
`module_tft`	Pilote de l'écran ILI9341 et du tactile XPT2046

platformio.ini

```
; PlatformIO Project Configuration File
; https://docs.platformio.org/page/projectconf.html

[env:esp32-c6-devkitc-1]
platform = https://github.com/pioarduino/platform-espressif32/releases/download/stable/platform-espressif32.zip
board = esp32-c6-devkitc-1
framework = arduino
monitor_speed = 115200
; WiFi + HTTPS + écran remplissent presque 1,3 Mo : on donne 3 Mo au programme
board_build.partitions = huge_app.csv

; Décommente ces deux lignes seulement si tu branches le câble sur le port "USB"
; de la carte (et pas sur le port "UART") et que le moniteur série reste muet.
;build_flags =
;    -D ARDUINO_USB_MODE=1
;    -D ARDUINO_USB_CDC_ON_BOOT=1

lib_deps =
    claws/BH1750
    adafruit/Adafruit BMP280 Library
    adafruit/Adafruit Unified Sensor
    adafruit/Adafruit BusIO
    adafruit/Adafruit GFX Library
    adafruit/Adafruit ILI9341
    paulstoffregen/XPT2046_Touchscreen
    miguelbalboa/MFRC522
    knolleary/PubSubClient
```

include/pinout.h

```
#ifndef PINOUT_H
#define PINOUT_H

// ============================================================
// PINOUT - ESP32-C6-WROOM
// ============================================================

// ---- Bus I2C (BH1750 + BMP280) ----
#define I2C_SDA     6
#define I2C_SCL     7

// ---- Bus SPI partagé (TFT + Tactile + RC522) ----
#define SPI_SCK     18
#define SPI_MOSI    19
#define SPI_MISO    20

// ---- Ecran TFT ----
#define TFT_CS      21
#define TFT_DC      22
#define TFT_RST     23
// LED (retroeclairage) -> 3V3 direct, pas de GPIO

// ---- Tactile (partage SPI_SCK / SPI_MOSI / SPI_MISO) ----
#define TOUCH_CS    4

// ---- RFID RC522 (partage SPI_SCK / SPI_MOSI / SPI_MISO) ----
#define RFID_CS     11
#define RFID_RST    0

// ---- Capteur MQ (analogique) ----
#define MQ_AO       1

// ---- Relais (via transistor, actif a l'etat haut) ----
#define RELAIS_PIN  10

// ---- LED RGB intégrée à la carte (WS2812) : couleur Tempo EDF ----
#define LED_RGB     8

// ---- Sortie PWM (gradation d'éclairage / vitesse VMC) ----
#define PWM_PIN     2

#endif // PINOUT_H
```

include/secrets_exemple.h

```h
#ifndef SECRETS_H
#define SECRETS_H

// =========================================================
// MODELE : copie ce fichier en "secrets.h" (même dossier)
// et remplis-le. secrets.h n'est jamais envoyé sur GitHub.
// =========================================================

// ---- WiFi de la maison (2,4 GHz, l'ESP32 ne voit pas le 5 GHz) ----
#define WIFI_SSID       "NomDeTaBox"
#define WIFI_MOTDEPASSE "MotDePasseWifi"

// ---- Broker MQTT = le Raspberry Pi qui fait tourner Home Assistant ----
// Mets l'adresse IP affichée sur l'écran du Pi (ex : 192.168.1.110).
#define MQTT_SERVEUR    "192.168.1.110"
#define MQTT_PORT       1883
// Le compte "esp32" créé dans Home Assistant (Paramètres > Personnes)
#define MQTT_UTILISATEUR "esp32"
#define MQTT_MOTDEPASSE  "MotDePasseEsp32"

#endif // SECRETS_H
```

src/main.cpp

```cpp
#include <WiFi.h>
// =========================================================
// Assistant domotique GouGouGaGAH - programme final
// (le programme de test des composants est dans GouGouGaGAH_DomotiqueESP32)
// =========================================================
#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>

#include "pinout.h"
#include "module_bh1750.h"
#include "module_bmp280.h"
#include "module_mq.h"
#include "module_rfid.h"
#include "module_relais.h"
#include "module_tft.h"
#include "module_menu.h"
#include "module_acces.h"
#include "module_pwm.h"
#include "module_tempo.h"
#include "module_ha.h"

void setup() {
  Serial.begin(115200);
  delay(1500); // laisse le temps au moniteur série de s'ouvrir
  Serial.println(F("=== Assistant domotique GouGouGaGAH ==="));

  // Bus I2C : BH1750 + BMP280
  Wire.begin(I2C_SDA, I2C_SCL);
  // Bus SPI partagé : TFT + tactile + RC522
  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI);

  // Toutes les puces SPI désélectionnées avant d'initialiser quoi que ce soit
  pinMode(TFT_CS, OUTPUT);   digitalWrite(TFT_CS, HIGH);
  pinMode(TOUCH_CS, OUTPUT); digitalWrite(TOUCH_CS, HIGH);
  pinMode(RFID_CS, OUTPUT);  digitalWrite(RFID_CS, HIGH);

  bh1750_setup();
  bmp280_setup();
  mq_setup();
  rfid_setup();
  relais_setup();
  pwm_setup();
  tempo_setup();
  acces_setup();  // badges autorisés lus en mémoire NVS
  tft_setup();
  menu_setup();   // écran tactile
  ha_setup();     // WiFi + MQTT vers Home Assistant
}

void loop() {
  acces_loop();   // badges + gâche de porte
  menu_loop();    // écran tactile
  ha_loop();      // Home Assistant
  tempo_loop();   // LED RGB Tempo (clignote en permanence)

  // Récapitulatif sur le moniteur série toutes les 5 s
  static unsigned long lastPrint = 0;
  if (millis() - lastPrint > 5000) {
    lastPrint = millis();
    Serial.println(String("Lux ") + bh1750_readLux() +
                   "  T " + bmp280_readTemperature() + "C" +
                   "  P " + bmp280_readPressure() + "hPa" +
                   "  MQ " + mq_readRaw() +
                   "  PWM " + pwm_get() + "%" +
                   "  MQTT " + (ha_estConnecte() ? "OK" : "--") +
                   "  WiFi " + (ha_wifiConnecte() ? String(WiFi.RSSI()) + "dBm" : String("--")));
  }
}
```

include/module_acces.h

```h
#ifndef MODULE_ACCES_H
#define MODULE_ACCES_H

#include <Arduino.h>

// Contrôle d'accès : badges autorisés (mémorisés en NVS) et gâche de porte (relais).

// ACCES_VALIDATION : avant d'ajouter un badge, il faut d'abord passer un badge déjà autorisé.
enum ModeAcces { ACCES_NORMAL, ACCES_VALIDATION, ACCES_AJOUT, ACCES_SUPPRESSION };

// Charge la liste des badges autorisés depuis la mémoire NVS.
void acces_setup();

// À appeler à chaque tour de loop() : lit les badges et referme la porte après le délai.
void acces_loop();

// Ouvre la porte (relais collé quelques secondes).
```

```h
void acces_ouvrirPorte();
bool acces_porteOuverte();

// Le prochain badge présenté sera ajouté / supprimé (15 s pour le présenter).
// Pour l'ajout, il faut d'abord passer un badge déjà autorisé (sauf si la liste est vide).
void acces_demanderAjout();
void acces_demanderSuppression();
void acces_annulerMode();
ModeAcces acces_mode();

// Efface tous les badges autorisés.
void acces_toutEffacer();

int acces_nbBadges();

// Dernier événement lisible (ex : "04 A3 F1 2B : autorise").
String acces_dernierMessage();

#endif // MODULE_ACCES_H
```

include/module_bh1750.h

```h
#ifndef MODULE_BH1750_H
#define MODULE_BH1750_H

#include <Arduino.h>

// Initialise le capteur BH1750 (I2C doit déjà être démarré avec Wire.begin()).
// Retourne true si le capteur a répondu correctement.
bool bh1750_setup();

// Retourne la dernière valeur de luminosité en lux.
// Retourne -1.0 si le capteur n'a pas été initialisé correctement.
float bh1750_readLux();

#endif // MODULE_BH1750_H
```

include/module_bmp280.h

```h
#ifndef MODULE_BMP280_H
#define MODULE_BMP280_H

#include <Arduino.h>

// Initialise le capteur BMP280 (I2C doit déjà être démarré avec Wire.begin()).
// Retourne true si le capteur a répondu correctement (adresse 0x76).
bool bmp280_setup();

// Retourne la dernière température lue en degrés Celsius.
// Retourne -999.0 si le capteur n'a pas été initialisé correctement.
float bmp280_readTemperature();

// Retourne la dernière pression lue en hPa.
// Retourne -1.0 si le capteur n'a pas été initialisé correctement.
float bmp280_readPressure();

#endif // MODULE_BMP280_H
```

include/module_ha.h

```h
#ifndef MODULE_HA_H
#define MODULE_HA_H

#include <Arduino.h>

// Liaison avec Home Assistant : WiFi + MQTT, avec découverte automatique
// (capteurs, porte, éclairage PWM et Tempo apparaissent tout seuls dans Home Assistant).
// Identifiants dans include/secrets.h (voir secrets_exemple.h).

// Lance la connexion WiFi (non bloquant).
void ha_setup();

// À appeler à chaque tour de loop() : garde WiFi + MQTT connectés,
// envoie les mesures toutes les 10 s et reçoit les ordres de Home Assistant.
void ha_loop();

// Signale un badge lu par le RC522 et ce qui en a été fait ("autorise", "refuse", "ajoute"...).
void ha_publierBadge(const String &uid, const String &resultat);

bool ha_wifiConnecte();
bool ha_estConnecte(); // connecté au broker MQTT

#endif // MODULE_HA_H
```

include/module_menu.h

```h
#ifndef MODULE_MENU_H
#define MODULE_MENU_H

#include <Arduino.h>

// Affiche le menu principal de test sur l'écran tactile
// (tft_setup() et les autres *_setup() doivent déjà avoir été appelés).
void menu_setup();

// À appeler à chaque tour de loop() : gère le tactile et rafraîchit l'écran affiché.
void menu_loop();

#endif // MODULE_MENU_H
```

include/module_mq.h

```h
#ifndef MODULE_MQ_H
#define MODULE_MQ_H

#include <Arduino.h>

// Configure la broche analogique du capteur MQ.
void mq_setup();

// Retourne la valeur brute lue (0-4095 sur l'ADC 12 bits de l'ESP32).
int mq_readRaw();
```

```

// Retourne la valeur en pourcentage (0-100%) de la plage ADC.
float mq_readPercent();

#endif // MODULE_MQ_H
```

include/module_pwm.h

```
#ifndef MODULE_PWM_H
#define MODULE_PWM_H

#include <Arduino.h>

// Sortie PWM (gradation d'éclairage, vitesse de VMC) sur PWM_PIN.

void pwm_setup();

// Règle le rapport cyclique en pourcentage (0 = éteint, 100 = plein).
void pwm_set(int pourcent);

int pwm_get();

#endif // MODULE_PWM_H
```

include/module_relais.h

```
#ifndef MODULE_RELAIS_H
#define MODULE_RELAIS_H

#include <Arduino.h>

// Configure la broche du relais et le met au repos (coupé).
void relais_setup();

// Colle (true) ou relâche (false) le relais.
void relais_set(bool actif);

// Inverse l'état du relais.
void relais_basculer();

// Retourne true si le relais est collé.
bool relais_estActif();

#endif // MODULE_RELAIS_H
```

include/module_rfid.h

```
#ifndef MODULE_RFID_H
#define MODULE_RFID_H

#include <Arduino.h>

// Initialise le lecteur RC522 (SPI.begin() doit déjà avoir été appelé).
// Retourne true si le module a répondu correctement.
bool rfid_setup();

// À appeler régulièrement dans loop(). Si une carte est détectée,
// retourne true et remplit uidStr avec l'UID en hexadécimal (ex: "04 A3 F1 2B").
bool rfid_checkCard(String &uidStr);

#endif // MODULE_RFID_H
```

include/module_tempo.h

```
#ifndef MODULE_TEMPO_H
#define MODULE_TEMPO_H

#include <Arduino.h>

// Couleur Tempo EDF du jour et du lendemain, affichée sur la LED RGB de la carte.
// La LED clignote en permanence : preuve que le programme tourne.

enum CouleurTempo { TEMPO_INCONNU = 0, TEMPO_BLEU = 1, TEMPO_BLANC = 2, TEMPO_ROUGE = 3 };

// Lance aussi une tâche de fond qui lit les couleurs sur internet toutes les heures
// (sans jamais bloquer le reste du programme).
void tempo_setup();

// À appeler à chaque tour de loop() : fait clignoter la LED.
void tempo_loop();

CouleurTempo tempo_aujourdhui();
CouleurTempo tempo_demain();

// "bleu", "blanc", "rouge" ou "inconnu"
const char *tempo_nom(CouleurTempo c);

#endif // MODULE_TEMPO_H
```

include/module_tft.h

```
#ifndef MODULE_TFT_H
#define MODULE_TFT_H

#include <Arduino.h>
#include <Adafruit_ILI9341.h>

// Initialise l'écran TFT ILI9341 et le tactile XPT2046
// (SPI.begin() doit déjà avoir été appelé).
void tft_setup();

// Affiche un écran de test (blocs de couleur + texte).
void tft_showTestScreen();

// Affiche une ligne de texte à une position donnée (petit helper).
void tft_printLine(int x, int y, const String &text);

// À appeler régulièrement dans loop(). Si l'écran est touché,
// remplit x/y avec les coordonnées brutes et retourne true.
bool tft_checkTouch(int &x, int &y);

// Comme tft_checkTouch, mais convertit en pixels écran (0..tft.width()-1, 0..tft.height()-1)
```

```
// avec la calibration enregistrée (voir tft_calibrerTactile).
bool tft_checkTouchPixel(int &x, int &y);

// Affiche 4 croix à toucher, calcule la calibration et l'enregistre en flash.
// Bloquant pendant la calibration. Retourne false si les points sont incohérents.
bool tft_calibrerTactile();

// true si une calibration a déjà été enregistrée.
bool tft_estCalibre();

// Accès direct à l'écran, pour les modules qui dessinent eux-mêmes (menu...).
Adafruit_ILI9341 &tft_get();

#endif // MODULE_TFT_H
```

src/module_acces.cpp

```
#include "module_acces.h"
#include "module_rfid.h"
#include "module_relais.h"
#include "module_ha.h"
#include <Preferences.h>

static const int MAX_BADGES = 20;
static const unsigned long DUREE_OUVERTURE = 3000; // ms de relais collé
static const unsigned long DUREE_MODE = 15000;     // ms pour présenter le badge à ajouter/supprimer

static Preferences nvs;
static String badges[MAX_BADGES];
static int nbBadges = 0;

static ModeAcces mode = ACCES_NORMAL;
static unsigned long debutMode = 0;
static unsigned long debutOuverture = 0;
static bool porteOuverte = false;
static String dernierMessage = "-";

// ------------------------------------------------------------ NVS ----

// Les badges sont rangés dans une seule chaîne : "04 A3 F1 2B;DE AD BE EF;..."
static void sauvegarder() {
  String liste;
  for (int i = 0; i < nbBadges; i++) {
    if (i) liste += ";";
    liste += badges[i];
  }
  nvs.putString("liste", liste);
}

static void charger() {
  String liste = nvs.getString("liste", "");
  nbBadges = 0;
  int debut = 0;
  while (debut < (int)liste.length() && nbBadges < MAX_BADGES) {
    int fin = liste.indexOf(';', debut);
    if (fin < 0) fin = liste.length();
    badges[nbBadges++] = liste.substring(debut, fin);
    debut = fin + 1;
  }
}

static int chercher(const String &uid) {
  for (int i = 0; i < nbBadges; i++) {
    if (badges[i] == uid) return i;
  }
  return -1;
}

// ------------------------------------------------------- traitement ----

static void signaler(const String &uid, const String &resultat) {
  dernierMessage = uid + " : " + resultat;
  Serial.println("[Acces] " + dernierMessage);
  ha_publierBadge(uid, resultat);
}

static void traiterBadge(const String &uid) {
  int pos = chercher(uid);
  switch (mode) {
    case ACCES_NORMAL:
      if (pos >= 0) {
        signaler(uid, "autorise");
        acces_ouvrirPorte();
      } else {
        signaler(uid, "refuse");
      }
      break;
    case ACCES_VALIDATION:
      if (pos >= 0) {
        signaler(uid, "valide, passe le nouveau badge");
        mode = ACCES_AJOUT;
        debutMode = millis(); // 15 s de plus pour le nouveau badge
      } else {
        signaler(uid, "refuse, ajout annule");
        mode = ACCES_NORMAL;
      }
      break;
    case ACCES_AJOUT:
      if (pos >= 0) signaler(uid, "deja connu");
      else if (nbBadges >= MAX_BADGES) signaler(uid, "liste pleine");
      else {
        badges[nbBadges++] = uid;
        sauvegarder();
        signaler(uid, "ajoute");
      }
      mode = ACCES_NORMAL;
      break;
    case ACCES_SUPPRESSION:
      if (pos < 0) signaler(uid, "inconnu");
      else {
        badges[pos] = badges[--nbBadges]; // le dernier prend sa place
        sauvegarder();
        signaler(uid, "supprime");
      }
      mode = ACCES_NORMAL;
      break;
  }
```

```
}

// ------------------------------------------------------------ public ----

void acces_setup() {
  nvs.begin("badges", false);
  charger();
  Serial.printf("[Acces] %d badge(s) autorise(s) en memoire\n", nbBadges);
}

void acces_loop() {
  String uid;
  if (rfid_checkCard(uid)) traiterBadge(uid);

  if (mode != ACCES_NORMAL && millis() - debutMode > DUREE_MODE) {
    mode = ACCES_NORMAL;
    dernierMessage = "Temps depasse";
  }
  if (porteOuverte && millis() - debutOuverture > DUREE_OUVERTURE) {
    porteOuverte = false;
    relais_set(false);
  }
}

void acces_ouvrirPorte() {
  porteOuverte = true;
  debutOuverture = millis();
  relais_set(true);
}

bool acces_porteOuverte() {
  return porteOuverte;
}

void acces_demanderAjout() {
  // Liste vide : personne ne peut valider, le premier badge s'enregistre directement
  mode = nbBadges == 0 ? ACCES_AJOUT : ACCES_VALIDATION;
  debutMode = millis();
}

void acces_demanderSuppression() {
  mode = ACCES_SUPPRESSION;
  debutMode = millis();
}

void acces_annulerMode() {
  mode = ACCES_NORMAL;
}

ModeAcces acces_mode() {
  return mode;
}

void acces_toutEffacer() {
  nbBadges = 0;
  sauvegarder();
  dernierMessage = "Tous les badges effaces";
}

int acces_nbBadges() {
  return nbBadges;
}

String acces_dernierMessage() {
  return dernierMessage;
}
```

src/module_bh1750.cpp

```
#include "module_bh1750.h"
#include <BH1750.h>

static BH1750 lightMeter;
static bool bh1750_ready = false;

bool bh1750_setup() {
  bh1750_ready = lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);
  if (bh1750_ready) {
    Serial.println(F("[BH1750] OK"));
  } else {
    Serial.println(F("[BH1750] ECHEC (verifier cablage I2C / ADDR->GND)"));
  }
  return bh1750_ready;
}

float bh1750_readLux() {
  if (!bh1750_ready) return -1.0;
  return lightMeter.readLightLevel();
}
```

src/module_bmp280.cpp

```
#include "module_bmp280.h"
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>

static Adafruit_BMP280 bmp;
static bool bmp280_ready = false;

bool bmp280_setup() {
  bmp280_ready = bmp.begin(0x76);
  if (bmp280_ready) {
    bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
                    Adafruit_BMP280::STANDBY_MS_500);
    Serial.println(F("[BMP280] OK"));
  } else {
    Serial.println(F("[BMP280] ECHEC (essayer 0x77, verifier CSB->3V3 / SDO->GND)"));
  }
  return bmp280_ready;
}

float bmp280_readTemperature() {
  if (!bmp280_ready) return -999.0;
```

```
  return bmp.readTemperature();
}

float bmp280_readPressure() {
  if (!bmp280_ready) return -1.0;
  return bmp.readPressure() / 100.0F; // Pa -> hPa
}
```

src/module_ha.cpp

```
#include "module_ha.h"
#include "module_bh1750.h"
#include "module_bmp280.h"
#include "module_mq.h"
#include "module_acces.h"
#include "module_pwm.h"
#include "module_tempo.h"
#include <WiFi.h>
#include <PubSubClient.h>

#if __has_include("secrets.h")
#include "secrets.h"
#else
#warning "include/secrets.h manquant : copie secrets_exemple.h en secrets.h et remplis-le"
#include "secrets_exemple.h"
#endif

// Tous les messages de la carte commencent par ce préfixe
#define BASE             "gougougagah"
#define TOPIC_DISPO      BASE "/dispo"          // online / offline
#define TOPIC_ETAT       BASE "/etat"           // mesures en JSON
#define TOPIC_PORTE      BASE "/porte"          // ON (ouverte) / OFF
#define TOPIC_PORTE_CMD  BASE "/porte/ouvrir"   // ordre d'ouverture venant de Home Assistant
#define TOPIC_PWM        BASE "/pwm"            // 0..100 %
#define TOPIC_PWM_CMD    BASE "/pwm/set"
#define TOPIC_BADGE      BASE "/badge"          // {"uid":...,"resultat":...}
#define TOPIC_HA_STATUS  "homeassistant/status" // Home Assistant vient de (re)démarrer

static const unsigned long PERIODE_MESURES = 10000; // ms entre deux envois
static const unsigned long PERIODE_RECONNEXION = 10000;

static WiFiClient wifiClient;
static PubSubClient mqtt(wifiClient);
static bool mqttAnnonce = false;
static unsigned long dernierEnvoi = 0;
static unsigned long derniereTentative = 0;
static bool wifiAnnonce = false;
static int portePubliee = -1; // -1 = jamais envoyé
static int pwmPublie = -1;

// ---------------------------------------------------------- découverte HA ----

// Bloc "appareil" commun : regroupe toutes les entités sous un seul appareil
static const char *APPAREIL =
  "\"dev\":{\"ids\":[\"" BASE "\"],\"name\":\"GouGouGaGAH\","
  "\"mf\":\"Witherwatch\",\"mdl\":\"ESP32-C6\"},"
  "\"avty_t\":\"" TOPIC_DISPO "\"";

static void annoncer(const char *type, const char *id, const String &champs) {
  String topic = String("homeassistant/") + type + "/" BASE "_" + id + "/config";
  String json = "{\"uniq_id\":\"" BASE "_" + String(id) + "\"," + champs + "," + APPAREIL + "}";
  mqtt.publish(topic.c_str(), json.c_str(), true);
}

// Capteur lu dans un champ JSON. mesure = true pour une valeur numérique (courbes + statistiques)
static void annoncerCapteur(const char *id, const char *nom, const char *topic, const char *champ,
                            bool mesure, const char *classe, const char *unite, const char *icone) {
  String c = String("\"name\":\"") + nom + "\",\"stat_t\":\"" + topic + "\","
             "\"val_tpl\":\"{{ value_json." + champ + " }}\"";
  if (mesure) c += ",\"stat_cla\":\"measurement\"";
  if (classe) c += String(",\"dev_cla\":\"") + classe + "\"";
  if (unite)  c += String(",\"unit_of_meas\":\"") + unite + "\"";
  if (icone)  c += String(",\"ic\":\"") + icone + "\"";
  annoncer("sensor", id, c);
}

static void annoncerTout() {
  annoncerCapteur("lux", "Luminosite", TOPIC_ETAT, "lux", true, "illuminance", "lx", nullptr);
  annoncerCapteur("temperature", "Temperature", TOPIC_ETAT, "temperature", true, "temperature", "°C", nullptr);
  annoncerCapteur("pression", "Pression", TOPIC_ETAT, "pression", true, "atmospheric_pressure", "hPa", nullptr);
  annoncerCapteur("gaz", "Gaz (brut)", TOPIC_ETAT, "gaz", true, nullptr, nullptr, "mdi:molecule");
  annoncerCapteur("badges", "Badges autorises", TOPIC_ETAT, "badges", true, nullptr, nullptr, "mdi:card-multiple");
  annoncerCapteur("tempo_jour", "Tempo aujourd'hui", TOPIC_ETAT, "tempo_jour", false, nullptr, nullptr, "mdi:calendar-today");
  annoncerCapteur("tempo_demain", "Tempo demain", TOPIC_ETAT, "tempo_demain", false, nullptr, nullptr, "mdi:calendar-arrow-right");
  annoncerCapteur("badge", "Dernier badge", TOPIC_BADGE, "uid", false, nullptr, nullptr, "mdi:card-account-details");
  annoncerCapteur("acces", "Dernier acces", TOPIC_BADGE, "resultat", false, nullptr, nullptr, "mdi:shield-key");

  annoncer("binary_sensor", "porte",
           "\"name\":\"Porte\",\"stat_t\":\"" TOPIC_PORTE "\",\"dev_cla\":\"door\"");
  annoncer("button", "ouvrir",
           "\"name\":\"Ouvrir la porte\",\"cmd_t\":\"" TOPIC_PORTE_CMD "\",\"ic\":\"mdi:door-open\"");
  annoncer("number", "pwm",
           "\"name\":\"Eclairage\",\"cmd_t\":\"" TOPIC_PWM_CMD "\",\"stat_t\":\"" TOPIC_PWM "\","
           "\"min\":0,\"max\":100,\"step\":5,\"unit_of_meas\":\"%\",\"mode\":\"slider\","
           "\"ic\":\"mdi:lightbulb-on-outline\"");

  Serial.println(F("[HA] Entites annoncees a Home Assistant"));
}

// ------------------------------------------------------------ envois ----

// Une valeur invalide (capteur absent) part en "null" : Home Assistant affiche "inconnu"
static String valeur(float v, bool valide, int decimales) {
  return valide ? String(v, decimales) : String("null");
}

static void envoyerMesures() {
  float lux = bh1750_readLux();
  float t = bmp280_readTemperature();
  float p = bmp280_readPressure();
  String json = String("{\"lux\":") + valeur(lux, lux >= 0, 1) +
                ",\"temperature\":" + valeur(t, t > -900, 1) +
                ",\"pression\":" + valeur(p, p >= 0, 1) +
                ",\"gaz\":" + mq_readRaw() +
                ",\"badges\":" + acces_nbBadges() +
                ",\"tempo_jour\":\"" + tempo_nom(tempo_aujourdhui()) + "\"" +
                ",\"tempo_demain\":\"" + tempo_nom(tempo_demain()) + "\"}";
```

```
  mqtt.publish(TOPIC_ETAT, json.c_str());
}

static void envoyerPorte() {
  portePubliee = acces_porteOuverte() ? 1 : 0;
  mqtt.publish(TOPIC_PORTE, portePubliee ? "ON" : "OFF", true);
}

static void envoyerPwm() {
  pwmPublie = pwm_get();
  mqtt.publish(TOPIC_PWM, String(pwmPublie).c_str(), true);
}

// ------------------------------------------------------------ réception ----

static void recevoir(char *topic, byte *payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];

  if (strcmp(topic, TOPIC_PORTE_CMD) == 0) {
    Serial.println(F("[HA] Ordre : ouvrir la porte"));
    acces_ouvrirPorte();
  } else if (strcmp(topic, TOPIC_PWM_CMD) == 0) {
    Serial.println("[HA] Eclairage : " + msg + " %");
    pwm_set(msg.toInt());
  } else if (strcmp(topic, TOPIC_HA_STATUS) == 0 && msg == "online") {
    annoncerTout(); // Home Assistant a redémarré : on se présente de nouveau
  }
}

// ------------------------------------------------------------ connexion ----

static void connecterMqtt() {
  Serial.printf("[HA] Connexion MQTT a %s:%d... ", MQTT_SERVEUR, MQTT_PORT);
  String id = String(BASE "-") + String((uint32_t)ESP.getEfuseMac(), HEX);
  // Testament : si la carte disparaît, le broker publie "offline" à sa place
  if (!mqtt.connect(id.c_str(), MQTT_UTILISATEUR, MQTT_MOTDEPASSE,
                    TOPIC_DISPO, 0, true, "offline")) {
    // 4/5 = mauvais identifiants, -2 = broker injoignable (IP ? Mosquitto démarré ?)
    Serial.printf("ECHEC (code %d)\n", mqtt.state());
    return;
  }
  Serial.println(F("OK"));
  mqtt.publish(TOPIC_DISPO, "online", true);
  annoncerTout();
  mqtt.subscribe(TOPIC_PORTE_CMD);
  mqtt.subscribe(TOPIC_PWM_CMD);
  mqtt.subscribe(TOPIC_HA_STATUS);
  envoyerPorte();
  envoyerPwm();
  envoyerMesures();
  dernierEnvoi = millis();
}

// ------------------------------------------------------------ public ----

void ha_setup() {
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  WiFi.setSleep(false); // sans ça le modem s'endort et le Pi coupe la connexion MQTT
  WiFi.begin(WIFI_SSID, WIFI_MOTDEPASSE);
  Serial.printf("[WiFi] Connexion a \"%s\"...\n", WIFI_SSID);

  mqtt.setServer(MQTT_SERVEUR, MQTT_PORT);
  mqtt.setCallback(recevoir);
  mqtt.setBufferSize(1024); // les messages de découverte dépassent les 256 octets par défaut
  mqtt.setSocketTimeout(3); // évite de figer l'écran trop longtemps si le Pi ne répond pas
  mqtt.setKeepAlive(30);    // plus tolérant si la boucle prend un peu de retard
  wifiClient.setConnectionTimeout(1500); // une tentative ratée ne bloque que 1,5 s
}

void ha_loop() {
  if (WiFi.status() != WL_CONNECTED) {
    if (wifiAnnonce) Serial.println(F("[WiFi] Connexion perdue"));
    wifiAnnonce = false;
    return;
  }
  if (!wifiAnnonce) {
    wifiAnnonce = true;
    Serial.print(F("[WiFi] OK, adresse IP "));
    Serial.println(WiFi.localIP());
    derniereTentative = 0; // tente MQTT tout de suite
  }

  if (!mqtt.connected()) {
    if (mqttAnnonce) {
      mqttAnnonce = false;
      Serial.printf("[HA] MQTT deconnecte (etat %d, WiFi %d dBm)\n", mqtt.state(), WiFi.RSSI());
    }
    if (derniereTentative == 0 || millis() - derniereTentative > PERIODE_RECONNEXION) {
      derniereTentative = millis();
      connecterMqtt();
    }
    return;
  }

  mqttAnnonce = true;
  mqtt.loop();
  // Changements faits depuis l'écran ou par un badge : on prévient tout de suite
  if (portePubliee != (acces_porteOuverte() ? 1 : 0)) envoyerPorte();
  if (pwmPublie != pwm_get()) envoyerPwm();
  if (millis() - dernierEnvoi > PERIODE_MESURES) {
    dernierEnvoi = millis();
    envoyerMesures();
  }
}

void ha_publierBadge(const String &uid, const String &resultat) {
  if (!mqtt.connected()) return;
  String json = "{\"uid\":\"" + uid + "\",\"resultat\":\"" + resultat + "\"}";
  mqtt.publish(TOPIC_BADGE, json.c_str(), true);
}

bool ha_wifiConnecte() {
  return WiFi.status() == WL_CONNECTED;
}

bool ha_estConnecte() {
  return mqtt.connected();
```

```
}
```

src/module_menu.cpp

```
#include "module_menu.h"
#include "module_tft.h"
#include "module_bh1750.h"
#include "module_bmp280.h"
#include "module_mq.h"
#include "module_acces.h"
#include "module_pwm.h"
#include "module_tempo.h"
#include "module_ha.h"
#include <WiFi.h>

// Pas d'accents à l'écran : la police par défaut d'Adafruit_GFX ne les affiche pas.

enum Ecran { MENU, CAPTEURS, PORTE, BADGES, ECLAIRAGE, INFOS, CALIBRER };

struct Bouton {
  int x, y, w, h;
  const char *texte;
  uint16_t couleur;
};

static const int NB_BOUTONS = 6;
static Bouton boutons[NB_BOUTONS];
static const Ecran cibles[NB_BOUTONS] = { CAPTEURS, PORTE, BADGES, ECLAIRAGE, INFOS, CALIBRER };
static Bouton retour;
static Bouton btnOuvrir;                                // écran Porte
static Bouton btnAjouter, btnSupprimer, btnEffacer;     // écran Badges
static Bouton btnMoins, btnPlus;                        // écran Eclairage

static Ecran ecranActuel = MENU;
static unsigned long dernierRafraichissement = 0;
static bool doigtPose = false;          // pour ne réagir qu'au moment où on pose le doigt
static unsigned long dernierContact = 0; // anti-rebond : relâché seulement après 150 ms sans contact
static unsigned long effacerArme = 0;   // "Tout effacer" demande un 2e appui dans les 3 s

// ------------------------------------------------------------- dessin ----

static void dessinerBouton(const Bouton &b) {
  Adafruit_ILI9341 &tft = tft_get();
  tft.fillRoundRect(b.x, b.y, b.w, b.h, 8, b.couleur);
  tft.drawRoundRect(b.x, b.y, b.w, b.h, 8, ILI9341_WHITE);
  tft.setTextSize(2);
  tft.setTextColor(ILI9341_WHITE);
  int largeurTexte = strlen(b.texte) * 12; // 6 px par caractère x taille 2
  tft.setCursor(b.x + (b.w - largeurTexte) / 2, b.y + (b.h - 16) / 2);
  tft.print(b.texte);
}

static bool estDans(const Bouton &b, int x, int y) {
  return x >= b.x && x < b.x + b.w && y >= b.y && y < b.y + b.h;
}

static void dessinerTitre(const char *titre) {
  Adafruit_ILI9341 &tft = tft_get();
  tft.fillScreen(ILI9341_BLACK);
  tft.fillRect(0, 0, tft.width(), 40, ILI9341_NAVY);
  tft.setTextSize(2);
  tft.setTextColor(ILI9341_WHITE);
  tft.setCursor(retour.x + retour.w + 10, 12);
  tft.print(titre);
  dessinerBouton(retour);
}

// Efface une zone puis y écrit une valeur (évite le scintillement d'un fillScreen).
static void ecrireValeur(int y, const String &texte, uint8_t taille, uint16_t couleur) {
  Adafruit_ILI9341 &tft = tft_get();
  tft.fillRect(0, y, tft.width(), 8 * taille + 4, ILI9341_BLACK);
  tft.setTextSize(taille);
  tft.setTextColor(couleur);
  tft.setCursor(10, y);
  tft.print(texte);
}

static uint16_t couleurTempo(CouleurTempo c) {
  switch (c) {
    case TEMPO_BLEU:  return ILI9341_BLUE;
    case TEMPO_BLANC: return ILI9341_WHITE;
    case TEMPO_ROUGE: return ILI9341_RED;
    default:          return ILI9341_DARKGREY;
  }
}

static void afficherMenu() {
  Adafruit_ILI9341 &tft = tft_get();
  tft.fillScreen(ILI9341_BLACK);
  tft.setTextSize(2);
  tft.setTextColor(ILI9341_GREEN);
  tft.setCursor(10, 12);
  tft.print("Assistant domotique");
  for (int i = 0; i < NB_BOUTONS; i++) dessinerBouton(boutons[i]);
}

static void afficherEcran(Ecran e) {
  ecranActuel = e;
  dernierRafraichissement = 0; // force un rafraîchissement immédiat

  switch (e) {
    case MENU:
      afficherMenu();
      break;
    case CAPTEURS:
      dessinerTitre("Capteurs");
      break;
    case PORTE:
      dessinerTitre("Porte");
      dessinerBouton(btnOuvrir);
      break;
    case BADGES:
      dessinerTitre("Badges");
      dessinerBouton(btnAjouter);
      dessinerBouton(btnSupprimer);
      dessinerBouton(btnEffacer);
      break;
```

```
    case ECLAIRAGE:
      dessinerTitre("Eclairage (PWM)");
      dessinerBouton(btnMoins);
      dessinerBouton(btnPlus);
      break;
    case INFOS:
      dessinerTitre("Infos");
      break;
    case CALIBRER:
      tft_calibrerTactile();
      doigtPose = true; // le stylet vient d'être levé : pas de clic fantôme
      afficherEcran(MENU);
      break;
  }
}

// ------------------------------------------------- mise à jour des écrans ----

static String texteLux() {
  float lux = bh1750_readLux();
  return lux < 0 ? String("non detecte") : String(lux, 1) + " lux";
}

static String texteTemperature() {
  float t = bmp280_readTemperature();
  return t < -900 ? String("non detecte") : String(t, 1) + " C";
}

static String textePression() {
  float p = bmp280_readPressure();
  return p < 0 ? String("non detecte") : String(p, 1) + " hPa";
}

static void dessinerJauge(int pourcent) {
  Adafruit_ILI9341 &tft = tft_get();
  int x = 20, y = 150, w = tft.width() - 40, h = 30;
  tft.drawRect(x, y, w, h, ILI9341_WHITE);
  int plein = (w - 4) * pourcent / 100;
  tft.fillRect(x + 2, y + 2, plein, h - 4, ILI9341_YELLOW);
  tft.fillRect(x + 2 + plein, y + 2, w - 4 - plein, h - 4, ILI9341_BLACK);
}

static void rafraichir() {
  switch (ecranActuel) {
    case CAPTEURS:
      ecrireValeur(55,  "Lumiere : " + texteLux(), 2, ILI9341_YELLOW);
      ecrireValeur(85,  "Temp.   : " + texteTemperature(), 2, ILI9341_ORANGE);
      ecrireValeur(115, "Pression: " + textePression(), 2, ILI9341_CYAN);
      ecrireValeur(145, "Gaz     : " + String(mq_readRaw()) + " / 4095", 2, ILI9341_GREEN);
      ecrireValeur(175, "Porte   : " + String(acces_porteOuverte() ? "ouverte" : "fermee"), 2, ILI9341_WHITE);
      ecrireValeur(205, "Eclair. : " + String(pwm_get()) + " %", 2, ILI9341_WHITE);
      break;
    case PORTE:
      ecrireValeur(160, acces_porteOuverte() ? "Porte OUVERTE" : "Porte fermee", 3,
                   acces_porteOuverte() ? ILI9341_GREEN : ILI9341_WHITE);
      ecrireValeur(205, acces_dernierMessage(), 1, ILI9341_LIGHTGREY);
      break;
    case BADGES: {
      ecrireValeur(55, String("Badges autorises : ") + acces_nbBadges(), 2, ILI9341_WHITE);
      String etat;
      uint16_t couleur = ILI9341_YELLOW;
      if (acces_mode() == ACCES_VALIDATION) etat = "Passe un badge autorise";
      else if (acces_mode() == ACCES_AJOUT) etat = "Passe le NOUVEAU badge";
      else if (acces_mode() == ACCES_SUPPRESSION) etat = "Passe le badge a enlever";
      else if (effacerArme && millis() - effacerArme < 3000) {
        etat = "Re-appuie : tout effacer";
        couleur = ILI9341_RED;
      } else {
        etat = acces_dernierMessage();
        couleur = ILI9341_LIGHTGREY;
      }
      ecrireValeur(150, etat, 2, couleur);
      break;
    }
    case ECLAIRAGE:
      ecrireValeur(110, String(pwm_get()) + " %", 3, ILI9341_YELLOW);
      dessinerJauge(pwm_get());
      break;
    case INFOS:
      ecrireValeur(50, ha_wifiConnecte() ? "WiFi : " + WiFi.localIP().toString() : String("WiFi : connexion..."),
                   2, ha_wifiConnecte() ? ILI9341_GREEN : ILI9341_ORANGE);
      ecrireValeur(80, ha_estConnecte() ? "Home Assistant : OK" : "Home Assistant : --",
                   2, ha_estConnecte() ? ILI9341_GREEN : ILI9341_ORANGE);
      ecrireValeur(120, String("Tempo jour  : ") + tempo_nom(tempo_aujourdhui()), 2,
                   couleurTempo(tempo_aujourdhui()));
      ecrireValeur(150, String("Tempo demain: ") + tempo_nom(tempo_demain()), 2,
                   couleurTempo(tempo_demain()));
      break;
    default:
      break;
  }
}

// Un appui (doigt qui vient de se poser) sur l'écran en cours
static void toucher(int x, int y) {
  if (ecranActuel == MENU) {
    for (int i = 0; i < NB_BOUTONS; i++) {
      if (estDans(boutons[i], x, y)) { afficherEcran(cibles[i]); return; }
    }
    return;
  }
  if (y < 40) { // toute la barre de titre sert de bouton retour
    acces_annulerMode();
    afficherEcran(MENU);
    return;
  }

  switch (ecranActuel) {
    case PORTE:
      if (estDans(btnOuvrir, x, y)) acces_ouvrirPorte();
      break;
    case BADGES:
      if (estDans(btnAjouter, x, y)) acces_demanderAjout();
      else if (estDans(btnSupprimer, x, y)) acces_demanderSuppression();
      else if (estDans(btnEffacer, x, y)) {
        if (effacerArme && millis() - effacerArme < 3000) {
          acces_toutEffacer();
          effacerArme = 0;
        } else {
```

```
          effacerArme = millis();
        }
      }
      break;
    case ECLAIRAGE:
      if (estDans(btnMoins, x, y)) pwm_set(pwm_get() - 10);
      else if (estDans(btnPlus, x, y)) pwm_set(pwm_get() + 10);
      break;
    default:
      break;
  }
  dernierRafraichissement = 0;
}

// ------------------------------------------------------------------ API ----

void menu_setup() {
  Adafruit_ILI9341 &tft = tft_get();
  const int W = tft.width(), marge = 8, haut = 40;

  // Menu : grille de 3 x 2 boutons
  int w = (W - 4 * marge) / 3;
  int h = (tft.height() - haut - 3 * marge) / 2;
  const char *textes[NB_BOUTONS] = { "Capteurs", "Porte", "Badges", "Eclair.", "Infos", "Calib." };
  uint16_t couleurs[NB_BOUTONS] = { ILI9341_DARKGREEN, ILI9341_ORANGE, ILI9341_PURPLE,
                                    ILI9341_OLIVE, ILI9341_NAVY, ILI9341_DARKGREY };
  for (int i = 0; i < NB_BOUTONS; i++) {
    int col = i % 3, ligne = i / 3;
    boutons[i] = { marge + col * (w + marge), haut + marge + ligne * (h + marge), w, h,
                   textes[i], couleurs[i] };
  }
  retour = { 4, 4, 70, 32, "<", ILI9341_DARKGREY };

  btnOuvrir    = { W / 2 - 100, 55, 200, 90, "OUVRIR", ILI9341_DARKGREEN };
  int wb = (W - 4 * marge) / 3;
  btnAjouter   = { marge,             85, wb, 50, "Ajouter", ILI9341_DARKGREEN };
  btnSupprimer = { 2 * marge + wb,    85, wb, 50, "Suppr.", ILI9341_MAROON };
  btnEffacer   = { 3 * marge + 2 * wb, 85, wb, 50, "Tout", ILI9341_DARKGREY };
  btnMoins     = { 20,          55, 80, 50, "-10", ILI9341_DARKGREY };
  btnPlus      = { W - 100,     55, 80, 50, "+10", ILI9341_DARKGREY };

  // Sans calibration, les boutons ne tombent pas sous le stylet : on calibre d'abord
  if (!tft_estCalibre()) {
    while (!tft_calibrerTactile()) {}
  }
  afficherEcran(MENU);
}

void menu_loop() {
  int x, y;
  bool touche = tft_checkTouchPixel(x, y);
  if (touche && !doigtPose) toucher(x, y);
  if (touche) {
    doigtPose = true;
    dernierContact = millis();
  } else if (millis() - dernierContact > 150) {
    doigtPose = false;
  }

  if (ecranActuel != MENU && millis() - dernierRafraichissement > 500) {
    dernierRafraichissement = millis();
    rafraichir();
  }
}
```

src/module_mq.cpp

```
#include "module_mq.h"
#include "pinout.h"

void mq_setup() {
  pinMode(MQ_AO, INPUT);
  Serial.println(F("[MQ] OK (lecture analogique prete)"));
}

int mq_readRaw() {
  return analogRead(MQ_AO);
}

float mq_readPercent() {
  return (analogRead(MQ_AO) / 4095.0f) * 100.0f;
}
```

src/module_pwm.cpp

```
#include "module_pwm.h"
#include "pinout.h"

static const uint32_t FREQUENCE = 5000; // Hz : pas de scintillement visible
static const uint8_t RESOLUTION = 8;    // bits -> valeurs 0..255
static int pourcentActuel = 0;

void pwm_setup() {
  ledcAttach(PWM_PIN, FREQUENCE, RESOLUTION);
  pwm_set(0);
  Serial.println(F("[PWM] OK (0 %)"));
}

void pwm_set(int pourcent) {
  pourcentActuel = constrain(pourcent, 0, 100);
  ledcWrite(PWM_PIN, pourcentActuel * 255 / 100);
}

int pwm_get() {
  return pourcentActuel;
}
```

src/module_relais.cpp

```
#include "module_relais.h"
#include "pinout.h"

static bool relais_actif = false;

void relais_setup() {
```

```
  pinMode(RELAIS_PIN, OUTPUT);
  relais_set(false);
  Serial.println(F("[Relais] OK (coupe au demarrage)"));
}

void relais_set(bool actif) {
  relais_actif = actif;
  digitalWrite(RELAIS_PIN, actif ? HIGH : LOW);
  Serial.println(actif ? F("[Relais] ON") : F("[Relais] OFF"));
}

void relais_basculer() {
  relais_set(!relais_actif);
}

bool relais_estActif() {
  return relais_actif;
}
```

src/module_rfid.cpp

```
#include "module_rfid.h"
#include "pinout.h"
#include <SPI.h>
#include <MFRC522.h>

static MFRC522 rfid(RFID_CS, RFID_RST);
static bool rfid_ready = false;

bool rfid_setup() {
  rfid.PCD_Init();
  byte version = rfid.PCD_ReadRegister(rfid.VersionReg);
  rfid_ready = (version != 0x00 && version != 0xFF);

  if (rfid_ready) {
    Serial.print(F("[RC522] OK (version registre 0x"));
    Serial.print(version, HEX);
    Serial.println(F(")"));
  } else {
    Serial.println(F("[RC522] ECHEC (verifier SPI et alim 3.3V, jamais 5V !)"));
  }
  return rfid_ready;
}

bool rfid_checkCard(String &uidStr) {
  if (!rfid_ready) return false;
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
    return false;
  }

  uidStr = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uidStr += "0";
    uidStr += String(rfid.uid.uidByte[i], HEX);
    if (i < rfid.uid.size - 1) uidStr += " ";
  }
  uidStr.toUpperCase();

  rfid.PICC_HaltA();
  return true;
}
```

src/module_tempo.cpp

```
#include "module_tempo.h"
#include "pinout.h"
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

// API publique gratuite (données RTE) : renvoie {"codeJour":1,...} avec 1 bleu, 2 blanc, 3 rouge.
// On essaie d'abord en http (léger, pas de chiffrement), puis en https si le serveur refuse.
#define URL_TEMPO_HTTP  "http://www.api-couleur-tempo.fr/api/jourTempo/"
#define URL_TEMPO_HTTPS "https://www.api-couleur-tempo.fr/api/jourTempo/"

static const unsigned long PERIODE_MAJ = 3600000UL;   // 1 h
static const unsigned long PERIODE_ESSAI = 300000UL;  // 5 min si la dernière requête a échoué
static const unsigned long CYCLE_LED = 2000;          // ms

// Écrites par la tâche de fond, lues par loop()
static volatile CouleurTempo aujourdhui = TEMPO_INCONNU;
static volatile CouleurTempo demain = TEMPO_INCONNU;
static int phaseAffichee = -1;

static CouleurTempo lireReponse(HTTPClient &http) {
  CouleurTempo c = TEMPO_INCONNU;
  if (http.GET() == HTTP_CODE_OK) {
    String json = http.getString();
    int pos = json.indexOf("\"codeJour\":");
    if (pos >= 0) {
      int code = json.substring(pos + 11).toInt();
      if (code >= 1 && code <= 3) c = (CouleurTempo)code;
    }
  }
  http.end();
  return c;
}

// Lit /today ou /tomorrow ; TEMPO_INCONNU si la requête échoue
static CouleurTempo lireJour(const char *jour) {
  WiFiClient client;
  HTTPClient http;
  http.setConnectTimeout(5000);
  http.setTimeout(5000);
  CouleurTempo c = TEMPO_INCONNU;
  if (http.begin(client, String(URL_TEMPO_HTTP) + jour)) c = lireReponse(http);
  if (c != TEMPO_INCONNU) return c;

  WiFiClientSecure clientSecure;
  clientSecure.setInsecure(); // pas de vérification du certificat : suffisant pour une couleur Tempo
  clientSecure.setHandshakeTimeout(10);
  if (http.begin(clientSecure, String(URL_TEMPO_HTTPS) + jour)) c = lireReponse(http);
  return c;
}

// Tâche de fond : les requêtes internet peuvent durer plusieurs secondes,
```

```
// elles ne doivent jamais bloquer l'écran, les badges ni Home Assistant.
static void tacheTempo(void *) {
  for (;;) {
    if (WiFi.status() != WL_CONNECTED) {
      vTaskDelay(pdMS_TO_TICKS(2000));
      continue;
    }
    aujourdhui = lireJour("today");
    demain = lireJour("tomorrow");
    Serial.printf("[Tempo] aujourd'hui %s, demain %s\n", tempo_nom(aujourdhui), tempo_nom(demain));
    unsigned long attente = aujourdhui != TEMPO_INCONNU ? PERIODE_MAJ : PERIODE_ESSAI;
    vTaskDelay(pdMS_TO_TICKS(attente));
  }
}

static void allumer(CouleurTempo c) {
  switch (c) {
    case TEMPO_BLEU:  rgbLedWrite(LED_RGB, 0, 0, 60); break;
    case TEMPO_BLANC: rgbLedWrite(LED_RGB, 40, 40, 40); break;
    case TEMPO_ROUGE: rgbLedWrite(LED_RGB, 60, 0, 0); break;
    default:          rgbLedWrite(LED_RGB, 30, 15, 0); break; // orange = couleur pas encore connue
  }
}

// Sur un cycle de 2 s : couleur du jour (longue), pause, couleur du lendemain (courte), pause
static void clignoter() {
  unsigned long t = millis() % CYCLE_LED;
  int phase = t < 700 ? 0 : t < 1000 ? 1 : t < 1300 ? 2 : 3;
  if (phase == phaseAffichee) return; // on n'écrit dans la LED qu'aux changements
  phaseAffichee = phase;
  if (phase == 0) allumer(aujourdhui);
  else if (phase == 2) allumer(demain);
  else rgbLedWrite(LED_RGB, 0, 0, 0);
}

void tempo_setup() {
  rgbLedWrite(LED_RGB, 0, 0, 0);
  xTaskCreate(tacheTempo, "tempo", 10240, nullptr, 1, nullptr);
  Serial.println(F("[Tempo] LED RGB prete"));
}

void tempo_loop() {
  clignoter();
}

CouleurTempo tempo_aujourdhui() {
  return aujourdhui;
}

CouleurTempo tempo_demain() {
  return demain;
}

const char *tempo_nom(CouleurTempo c) {
  switch (c) {
    case TEMPO_BLEU:  return "bleu";
    case TEMPO_BLANC: return "blanc";
    case TEMPO_ROUGE: return "rouge";
    default:          return "inconnu";
  }
}
```

src/module_tft.cpp

```
#include "module_tft.h"
#include "pinout.h"
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <XPT2046_Touchscreen.h>
#include <Preferences.h>
#include <algorithm>

// Ton module n'est pas un vrai ILI9341 : sa mémoire est organisée nativement en
// 320 x 240 (le grand côté sert de lignes), et il ignore l'échange lignes/colonnes
// utilisé d'habitude pour passer en paysage. On garde donc l'adressage "portrait"
// du contrôleur, mais en lui donnant une taille logique de 320 x 240.
class EcranClone : public Adafruit_ILI9341 {
public:
  using Adafruit_ILI9341::Adafruit_ILI9341;
  void forcerTaille(int16_t w, int16_t h) {
    _width = w;
    _height = h;
  }
};

static EcranClone tft(TFT_CS, TFT_DC, TFT_RST);
static XPT2046_Touchscreen touch(TOUCH_CS); // pas d'IRQ, lecture en polling

// ---- Calibration du tactile ----
// Valeurs brutes lues aux bords gauche/droite (X) et haut/bas (Y) de l'écran.
// Un MIN plus grand que le MAX veut dire que l'axe est inversé (map() le gère).
// "echange" : l'axe X de l'écran correspond à l'axe Y brut du tactile.
// Ces valeurs par défaut sont remplacées par celles de l'écran "Calibrer",
// enregistrées en mémoire flash (elles survivent au redémarrage et au reflash).
struct CalibTactile {
  int xMin, xMax, yMin, yMax;
  bool echange;
};
static CalibTactile calib = { 300, 3800, 300, 3800, false };
static bool calibre = false;

static const int TOUCH_PRESSION_MIN = 400; // en dessous, appui trop léger : ignoré

static void tft_chargerCalibration() {
  Preferences prefs;
  prefs.begin("tactile", true);
  calibre = prefs.isKey("xMin");
  if (calibre) {
    calib.xMin = prefs.getInt("xMin");
    calib.xMax = prefs.getInt("xMax");
    calib.yMin = prefs.getInt("yMin");
    calib.yMax = prefs.getInt("yMax");
    calib.echange = prefs.getBool("echange");
    Serial.printf("[Tactile] Calibration chargee : X %d..%d  Y %d..%d  echange=%d\n",
                  calib.xMin, calib.xMax, calib.yMin, calib.yMax, calib.echange);
  } else {
```

```
    Serial.println(F("[Tactile] Pas encore calibre : menu > Calibrer"));
  }
  prefs.end();
}

static void tft_enregistrerCalibration() {
  Preferences prefs;
  prefs.begin("tactile", false);
  prefs.putInt("xMin", calib.xMin);
  prefs.putInt("xMax", calib.xMax);
  prefs.putInt("yMin", calib.yMin);
  prefs.putInt("yMax", calib.yMax);
  prefs.putBool("echange", calib.echange);
  prefs.end();
}

// Lit un registre de l'écran (lecture SPI générique, valable pour ILI9341 et ST7789).
// Le premier octet renvoyé par l'écran est un octet "fantôme", d'où index + 1.
static uint8_t tft_lireRegistre(uint8_t registre, uint8_t index) {
  return tft.Adafruit_SPITFT::readcommand8(registre, index + 1);
}

// Affiche sur le moniteur série l'identifiant de l'écran et l'orientation réellement
// appliquée, pour savoir quel contrôleur se cache derrière le module.
static void tft_diagnostic() {
  Serial.print(F("[TFT] ID ecran (0x04) : "));
  for (uint8_t i = 0; i < 3; i++) {
    Serial.print(tft_lireRegistre(0x04, i), HEX);
    Serial.print(' ');
  }
  Serial.print(F(" | ID4 (0xD3) : "));
  for (uint8_t i = 0; i < 3; i++) {
    Serial.print(tft_lireRegistre(0xD3, i), HEX);
    Serial.print(' ');
  }
  Serial.print(F(" | MADCTL lu : 0x"));
  Serial.print(tft_lireRegistre(0x0B, 0), HEX);
  Serial.println();
}

// ---- Test des orientations au démarrage ----
// Chaque réglage s'affiche 5 s avec une grande lettre. Note la lettre où le cadre blanc
// fait tout le tour de l'écran, où la lettre se lit normalement (pas en miroir) et où
// les carrés R V B sont dans cet ordre, en bas. Mets ensuite TFT_TEST_ORIENTATIONS à 0
// et TFT_CONFIG à cette lettre.
#define TFT_TEST_ORIENTATIONS 0
#define TFT_CONFIG 'H' // choisi d'après le test : rendu correct sur ce module

struct ConfigEcran {
  char lettre;
  uint8_t madctl; // valeur envoyée au registre 0x36 (RGB, sans échange lignes/colonnes)
  const char *description;
};

static const ConfigEcran configs[] = {
  { 'F', 0x00, "320x240, normal" },
  { 'G', 0x40, "320x240, miroir gauche/droite" },
  { 'H', 0x80, "320x240, miroir haut/bas" },
  { 'I', 0xC0, "320x240, retourne (180 degres)" },
};
static const int NB_CONFIGS = sizeof(configs) / sizeof(configs[0]);

static void tft_appliquerConfig(const ConfigEcran &cfg) {
  tft.setRotation(0); // remet l'adressage de base du contrôleur
  uint8_t m = cfg.madctl;
  tft.sendCommand(0x36, &m, 1);
  tft.forcerTaille(320, 240);
  tft.fillScreen(ILI9341_BLACK); // efface toute la mémoire (320 x 240)
}

static void tft_dessinerMire(char lettre) {
  int w = tft.width(), h = tft.height();
  tft.fillScreen(ILI9341_BLACK);
  tft.drawRect(0, 0, w, h, ILI9341_WHITE);
  tft.drawRect(2, 2, w - 4, h - 4, ILI9341_WHITE);
  tft.setTextColor(ILI9341_WHITE);
  tft.setTextSize(8);
  tft.setCursor(w / 2 - 24, 20);
  tft.print(lettre);
  // Trois carrés : R doit être rouge, V vert, B bleu
  const uint16_t couleurs[3] = { ILI9341_RED, ILI9341_GREEN, ILI9341_BLUE };
  const char lettres[3] = { 'R', 'V', 'B' };
  for (int i = 0; i < 3; i++) {
    int x = w / 2 - 75 + i * 52;
    tft.fillRect(x, h - 70, 46, 46, couleurs[i]);
    tft.setTextSize(3);
    tft.setTextColor(ILI9341_WHITE);
    tft.setCursor(x + 14, h - 58);
    tft.print(lettres[i]);
  }
}

static void tft_testOrientations() {
  for (int i = 0; i < NB_CONFIGS; i++) {
    tft_appliquerConfig(configs[i]);
    tft_dessinerMire(configs[i].lettre);
    Serial.printf("[TFT] Test %c : %s\n", configs[i].lettre, configs[i].description);
    delay(5000);
  }
}

void tft_setup() {
  // 20 MHz au lieu de 40 MHz par défaut : plus fiable avec des fils sur plaque à trous
  tft.begin(20000000);
  if (TFT_TEST_ORIENTATIONS) tft_testOrientations();
  for (int i = 0; i < NB_CONFIGS; i++) {
    if (configs[i].lettre == TFT_CONFIG) tft_appliquerConfig(configs[i]);
  }
  tft_diagnostic();
  tft.fillScreen(ILI9341_BLACK);
  touch.begin();
  touch.setRotation(0); // axes bruts : la calibration gère sens et échange des axes
  tft_chargerCalibration();
  Serial.println(F("[TFT] OK (verifier visuellement l'ecran)"));
  Serial.println(F("[Tactile] OK (touche l'ecran pour tester)"));
}

void tft_showTestScreen() {
  tft.fillScreen(ILI9341_BLACK);
```

```
  tft.setCursor(10, 10);
  tft.setTextColor(ILI9341_GREEN);
  tft.setTextSize(2);
  tft.println("Diagnostic OK");

  tft.setTextSize(1);
  tft.setTextColor(ILI9341_WHITE);
  tft.setCursor(10, 40);
  tft.println("Touche l'ecran pour");
  tft.setCursor(10, 55);
  tft.println("tester le tactile.");

  tft.fillRect(10, 100, 40, 30, ILI9341_RED);
  tft.fillRect(60, 100, 40, 30, ILI9341_GREEN);
  tft.fillRect(110, 100, 40, 30, ILI9341_BLUE);
  tft.drawRect(160, 100, 40, 30, ILI9341_YELLOW);
}

void tft_printLine(int x, int y, const String &text) {
  tft.fillRect(x, y, 220, 12, ILI9341_BLACK); // efface la ligne précédente
  tft.setCursor(x, y);
  tft.setTextColor(ILI9341_WHITE);
  tft.setTextSize(1);
  tft.print(text);
}

// Le premier échantillon d'un appui est souvent faux (stylet pas encore bien posé) :
// on lit plusieurs points, on garde ceux assez appuyés et on prend la médiane.
bool tft_checkTouch(int &x, int &y) {
  if (!touch.touched()) return false;
  const int N = 7;
  int xs[N], ys[N], n = 0;
  for (int i = 0; i < N; i++) {
    TS_Point p = touch.getPoint();
    if (p.z >= TOUCH_PRESSION_MIN) {
      xs[n] = p.x;
      ys[n] = p.y;
      n++;
    }
    delay(2);
  }
  if (n < 4) return false; // appui trop léger ou instable
  std::sort(xs, xs + n);
  std::sort(ys, ys + n);
  x = xs[n / 2];
  y = ys[n / 2];
  return true;
}

bool tft_checkTouchPixel(int &x, int &y) {
  int rx, ry;
  if (!tft_checkTouch(rx, ry)) return false;
  int a = calib.echange ? ry : rx;
  int b = calib.echange ? rx : ry;
  x = constrain(map(a, calib.xMin, calib.xMax, 0, tft.width() - 1), 0, tft.width() - 1);
  y = constrain(map(b, calib.yMin, calib.yMax, 0, tft.height() - 1), 0, tft.height() - 1);
  return true;
}

// ---- Écran de calibration ----

static const int CALIB_MARGE = 20; // distance des croix aux bords

static void tft_dessinerCroix(int x, int y, uint16_t couleur) {
  tft.drawFastHLine(x - 10, y, 21, couleur);
  tft.drawFastVLine(x, y - 10, 21, couleur);
  tft.drawCircle(x, y, 5, couleur);
}

// Attend un appui franc, en fait la moyenne pendant ~300 ms, puis attend qu'on relâche.
static void tft_lireAppuiCalibration(int &rx, int &ry) {
  long sx, sy;
  int n;
  do {
    while (!touch.touched()) delay(10);
    delay(80); // laisse le stylet se poser
    sx = sy = n = 0;
    unsigned long debut = millis();
    while (millis() - debut < 300) {
      int x, y;
      if (tft_checkTouch(x, y)) { sx += x; sy += y; n++; }
    }
  } while (n < 5);
  rx = sx / n;
  ry = sy / n;
  unsigned long relache = millis();
  while (millis() - relache < 200) { // relâché depuis 200 ms
    if (touch.touched()) relache = millis();
    delay(10);
  }
}

bool tft_calibrerTactile() {
  const int w = tft.width(), h = tft.height(), m = CALIB_MARGE;
  // Ordre : haut-gauche, haut-droite, bas-droite, bas-gauche
  const int cx[4] = { m, w - 1 - m, w - 1 - m, m };
  const int cy[4] = { m, m, h - 1 - m, h - 1 - m };
  int rx[4], ry[4];

  for (int i = 0; i < 4; i++) {
    tft.fillScreen(ILI9341_BLACK);
    tft.setTextSize(2);
    tft.setTextColor(ILI9341_WHITE);
    tft.setCursor(40, h / 2 - 20);
    tft.print("Appuie au centre");
    tft.setCursor(40, h / 2 + 4);
    tft.printf("de la croix (%d/4)", i + 1);
    tft_dessinerCroix(cx[i], cy[i], ILI9341_YELLOW);
    tft_lireAppuiCalibration(rx[i], ry[i]);
    Serial.printf("[Calib] point %d (%d,%d) : brut x=%d y=%d\n", i + 1, cx[i], cy[i], rx[i], ry[i]);
  }

  // Quel axe brut varie quand on va de gauche à droite ?
  int dxBrutX = abs((rx[1] + rx[2]) - (rx[0] + rx[3]));
  int dxBrutY = abs((ry[1] + ry[2]) - (ry[0] + ry[3]));
  bool echange = dxBrutY > dxBrutX;
  int a[4], b[4];
  for (int i = 0; i < 4; i++) {
    a[i] = echange ? ry[i] : rx[i];
```

```
    b[i] = echange ? rx[i] : ry[i];
  }
  // Moyenne des deux croix de chaque bord, puis extrapolation jusqu'au bord de l'écran
  float gauche = (a[0] + a[3]) / 2.0f, droite = (a[1] + a[2]) / 2.0f;
  float haut = (b[0] + b[1]) / 2.0f, bas = (b[2] + b[3]) / 2.0f;
  float parPixelX = (droite - gauche) / (w - 1 - 2 * m);
  float parPixelY = (bas - haut) / (h - 1 - 2 * m);

  bool ok = fabsf(droite - gauche) > 800 && fabsf(bas - haut) > 600;
  tft.fillScreen(ILI9341_BLACK);
  tft.setTextSize(2);
  tft.setCursor(20, h / 2 - 10);
  if (!ok) {
    Serial.println(F("[Calib] ECHEC : points trop proches, recommence en appuyant bien sur les croix"));
    tft.setTextColor(ILI9341_RED);
    tft.print("Echec, recommence");
    delay(1500);
    return false;
  }
  calib.xMin = lroundf(gauche - m * parPixelX);
  calib.xMax = lroundf(droite + m * parPixelX);
  calib.yMin = lroundf(haut - m * parPixelY);
  calib.yMax = lroundf(bas + m * parPixelY);
  calib.echange = echange;
  tft_enregistrerCalibration();
  calibre = true;
  Serial.printf("[Calib] OK : X %d..%d  Y %d..%d  echange=%d (enregistre)\n",
                calib.xMin, calib.xMax, calib.yMin, calib.yMax, calib.echange);
  tft.setTextColor(ILI9341_GREEN);
  tft.print("Calibration OK");
  delay(1000);
  return true;
}

bool tft_estCalibre() {
  return calibre;
}

Adafruit_ILI9341 &tft_get() {
  return tft;
}
```

Annexe B. Codes et configuration Home Assistant

B.1 Installation et intégration MQTT

Élément	Configuration
Serveur	Raspberry Pi 3, Home Assistant OS 18.4, en WiFi 2,4 GHz, adresse fixe 192.168.1.110, interface web `http://192.168.1.110:8123`
Broker MQTT	Module complémentaire **Mosquitto broker** (configuration par défaut, port 1883)
Compte de la carte	Utilisateur Home Assistant `esp32` (Paramètres → Personnes), utilisé par la carte pour se connecter au broker
Intégration	**MQTT** (Paramètres → Appareils et services), découverte activée, préfixe `homeassistant`
Accès distant	Par VPN vers le réseau local de l'habitation

B.2 Messages MQTT échangés

Topic	Sens	Contenu
`gougougagah/etat`	ESP32 → HA	JSON des mesures toutes les 10 s, ex. : `{"lux":215.8,"temperature":21.4,"pression":1012.6,"gaz":812,"badges":2,"tempo_jour":"BLEU","tempo_demain":"`
`gougougagah/porte`	ESP32 → HA	`ON` (ouverte) / `OFF`
`gougougagah/porte/ouvrir`	HA → ESP32	ordre d'ouverture (relais collé 3 s)
`gougougagah/pwm`, `…/pwm/set`	les deux	rapport cyclique 0 à 100
`gougougagah/badge`	ESP32 → HA	`{"uid":"A1B2C3D4","resultat":"autorise"}`
`gougougagah/ordre`	HA → ESP32	ordre texte d'une automatisation (pergola, volets…), affiché au moniteur série et sur l'écran Infos
`gougougagah/dispo`	ESP32 → HA	`online` / `offline` (testament MQTT)
`homeassistant/status`	HA → ESP32	`online` au redémarrage de HA : la carte renvoie ses annonces

B.3 Entités créées par découverte MQTT

À chaque connexion, la carte publie (message retenu) une annonce sur `homeassistant/<type>/gougougagah_<id>/config`. Aucune ligne de YAML n'est donc nécessaire pour créer les entités. Exemple d'annonce pour la température :

```
{
  "uniq_id": "gougougagah_temperature",
  "name": "Temperature",
  "stat_t": "gougougagah/etat",
  "val_tpl": "{{ value_json.temperature }}",
  "stat_cla": "measurement",
  "dev_cla": "temperature",
  "unit_of_meas": "°C",
  "dev": {
    "ids": [
      "gougougagah"
    ],
    "name": "GouGouGaGAH",
    "mf": "Witherwatch",
    "mdl": "ESP32-C6"
  },
  "avty_t": "gougougagah/dispo"
}
```

Entité	Grandeur	Source MQTT
`sensor.gougougagah_luminosite`	Luminosité (lx)	`gougougagah/etat → lux`
`sensor.gougougagah_temperature`	Température (°C)	`gougougagah/etat → temperature`
`sensor.gougougagah_pression`	Pression (hPa)	`gougougagah/etat → pression`
`sensor.gougougagah_gaz_brut`	Gaz (valeur brute 0-4095)	`gougougagah/etat → gaz`
`sensor.gougougagah_badges_autorises`	Nombre de badges autorisés	`gougougagah/etat → badges`
`sensor.gougougagah_tempo_aujourd_hui`	Couleur Tempo du jour	`gougougagah/etat → tempo_jour`
`sensor.gougougagah_tempo_demain`	Couleur Tempo du lendemain	`gougougagah/etat → tempo_demain`
`sensor.gougougagah_dernier_badge`	UID du dernier badge présenté	`gougougagah/badge → uid`
`sensor.gougougagah_dernier_acces`	Résultat (autorise, refuse, ajoute…)	`gougougagah/badge → resultat`
`binary_sensor.gougougagah_porte`	Porte ouverte / fermée	`gougougagah/porte (ON / OFF)`
`button.gougougagah_ouvrir_la_porte`	Bouton d'ouverture de la porte	`gougougagah/porte/ouvrir`
`number.gougougagah_eclairage`	Curseur PWM 0-100 %	`gougougagah/pwm/set, état gougougagah/pwm`

B.4 Tableau de bord, automatisations et scripts (YAML)

`automations.yaml`

```
# Automatisations AssistDomo (Paramètres > Automatisations et scènes)
# Chacune peut être collée dans : Créer une automatisation > ⋮ > Modifier en YAML.

# 1. Badge RFID présenté -> prévenir l'utilisateur
- id: assistdomo_badge
  alias: "AssistDomo - Badge RFID detecte"
```

```
  description: "Notification à chaque badge présenté au lecteur RC522"
  triggers:
    - trigger: mqtt
      topic: gougougagah/badge
  actions:
    - action: persistent_notification.create
      data:
        title: "Badge RFID"
        message: >-
          Badge {{ trigger.payload_json.uid }} :
          {{ trigger.payload_json.resultat }}
          ({{ now().strftime('%H:%M:%S') }})
    - action: mqtt.publish
      data:
        topic: gougougagah/ordre
        payload: "NOTIF : badge {{ trigger.payload_json.uid }} {{ trigger.payload_json.resultat }}"
  mode: queued

# 2. Pergola bioclimatique / volets : température mesurée vs consigne, selon la saison
- id: assistdomo_pergola
  alias: "AssistDomo - Pergola et volets selon temperature"
  description: "Compare la température du BMP280 à la consigne et envoie l'ordre à l'ESP32"
  triggers:
    - trigger: numeric_state
      entity_id: sensor.gougougagah_temperature
      above: input_number.temperature_souhaitee
    - trigger: numeric_state
      entity_id: sensor.gougougagah_temperature
      below: input_number.temperature_souhaitee
    - trigger: state
      entity_id: input_number.temperature_souhaitee
  variables:
    t: "{{ states('sensor.gougougagah_temperature') | float(0) }}"
    consigne: "{{ states('input_number.temperature_souhaitee') | float(20) }}"
    saison: >-
      {% set m = now().month %}
      {{ 'ete' if m in [6,7,8] else 'hiver' if m in [12,1,2]
         else 'printemps' if m in [3,4,5] else 'automne' }}
  actions:
    - choose:
        - conditions: "{{ t > consigne }}"
          sequence:
            - action: mqtt.publish
              data:
                topic: gougougagah/ordre
                payload: >-
                  {{ 'PERGOLA : lames fermees + VOLETS baisses'
                     if saison == 'ete' else 'PERGOLA : lames fermees' }}
                  ({{ saison }}, T={{ t }} > {{ consigne }})
      default:
        - action: mqtt.publish
          data:
            topic: gougougagah/ordre
            payload: >-
              {{ 'PERGOLA : lames ouvertes + VOLETS ouverts'
                 if saison in ['hiver','automne'] else 'PERGOLA : lames ouvertes' }}
              ({{ saison }}, T={{ t }} <= {{ consigne }})
  mode: restart

# 3. Éclairage automatique selon la luminosité (sortie PWM)
- id: assistdomo_eclairage_nuit
  alias: "AssistDomo - Eclairage si sombre"
  triggers:
    - trigger: numeric_state
      entity_id: sensor.gougougagah_luminosite
      below: 30
  actions:
    - action: number.set_value
      target:
        entity_id: number.gougougagah_eclairage
      data:
        value: 80
    - action: mqtt.publish
      data:
        topic: gougougagah/ordre
        payload: "ECLAIRAGE : allume a 80 % ({{ states('sensor.gougougagah_luminosite') }} lx)"
  mode: single

- id: assistdomo_eclairage_jour
  alias: "AssistDomo - Extinction si lumineux"
  triggers:
    - trigger: numeric_state
      entity_id: sensor.gougougagah_luminosite
      above: 100
  actions:
    - action: number.set_value
      target:
        entity_id: number.gougougagah_eclairage
      data:
        value: 0
    - action: mqtt.publish
      data:
        topic: gougougagah/ordre
        payload: "ECLAIRAGE : eteint ({{ states('sensor.gougougagah_luminosite') }} lx)"
  mode: single

# 4. Jour Tempo rouge demain -> prévenir la veille
- id: assistdomo_tempo_rouge
  alias: "AssistDomo - Alerte Tempo rouge demain"
  triggers:
    - trigger: state
      entity_id: sensor.gougougagah_tempo_demain
      to: "rouge"
  actions:
    - action: persistent_notification.create
      data:
        title: "Tempo EDF"
        message: "Demain est un jour ROUGE : limiter le chauffage et les gros appareils."
    - action: mqtt.publish
      data:
        topic: gougougagah/ordre
        payload: "TEMPO : demain rouge, delestage prevu"
  mode: single
```

configuration.yaml

```
# Ajout à configuration.yaml (facultatif : la même entrée peut être créée
```

```yaml
# dans Paramètres > Appareils et services > Entrées > Créer une entrée > Nombre).
# Les entités de l'ESP32 n'ont besoin d'aucune ligne ici : elles arrivent
# par la découverte MQTT (intégration MQTT + broker Mosquitto).

input_number:
  temperature_souhaitee:
    name: Température souhaitée
    min: 15
    max: 30
    step: 0.5
    unit_of_measurement: "°C"
    mode: slider
    icon: mdi:thermometer-check
```

dashboard.yaml

```yaml
# Tableau de bord AssistDomo
# Paramètres > Tableaux de bord > Ajouter un tableau de bord > Nouveau tableau de bord vide
# puis ✏️ > ⋮ > Éditeur de configuration brute : coller ce fichier.
title: AssistDomo
views:
  - title: Maison
    path: maison
    icon: mdi:home-automation
    cards:
      - type: entities
        title: Mesures en direct
        show_header_toggle: false
        entities:
          - entity: sensor.gougougagah_temperature
            name: Température
          - entity: sensor.gougougagah_luminosite
            name: Luminosité
          - entity: sensor.gougougagah_pression
            name: Pression
          - entity: sensor.gougougagah_gaz_brut
            name: Gaz (MQ, brut)

      - type: vertical-stack
        cards:
          - type: entity
            entity: binary_sensor.gougougagah_porte
            name: Porte (gâche)
          - type: button
            entity: button.gougougagah_ouvrir_la_porte
            name: Ouvrir la porte
            icon: mdi:door-open
            show_state: false
            tap_action:
              action: perform-action
              perform_action: button.press
              target:
                entity_id: button.gougougagah_ouvrir_la_porte

      - type: entities
        title: Commandes
        show_header_toggle: false
        entities:
          - entity: number.gougougagah_eclairage
            name: Éclairage (PWM)
          - entity: input_number.temperature_souhaitee
            name: Température souhaitée

      - type: history-graph
        title: Température (24 h)
        hours_to_show: 24
        entities:
          - entity: sensor.gougougagah_temperature
            name: Température
          - entity: input_number.temperature_souhaitee
            name: Consigne

      - type: history-graph
        title: Luminosité (24 h)
        hours_to_show: 24
        entities:
          - entity: sensor.gougougagah_luminosite
            name: Luminosité

      - type: history-graph
        title: Pression (24 h)
        hours_to_show: 24
        entities:
          - entity: sensor.gougougagah_pression
            name: Pression

      - type: history-graph
        title: Gaz MQ (24 h)
        hours_to_show: 24
        entities:
          - entity: sensor.gougougagah_gaz_brut
            name: Gaz (brut)

      - type: entities
        title: Contrôle d'accès RFID
        show_header_toggle: false
        entities:
          - entity: sensor.gougougagah_dernier_badge
            name: Dernier badge
          - entity: sensor.gougougagah_dernier_acces
            name: Résultat
          - entity: sensor.gougougagah_badges_autorises
            name: Badges enregistrés

      - type: logbook
        title: Historique porte et badges
        hours_to_show: 24
        entities:
          - binary_sensor.gougougagah_porte
          - sensor.gougougagah_dernier_badge
          - sensor.gougougagah_dernier_acces

      - type: entities
        title: Tempo EDF
        show_header_toggle: false
        entities:
          - entity: sensor.gougougagah_tempo_aujourd_hui
```

```
102             name: Aujourd'hui
103           - entity: sensor.gougougagah_tempo_demain
104             name: Demain
```